Conversiono

AI-Powered Behavioral Intelligence for E-Commerce

MASTER TECHNICAL DOCUMENT

Complete Production Engineering Reference

Version 1.0 · March 2026 · Confidential

Product	Conversiono — AI-Powered Behavioral Intelligence for E-Commerce
Document type	Master Technical Reference — Architecture, Implementation, Scaling, Sprints
Audience	Technical co-founder, engineering team, future technical hires
Scope	Full production system: architecture, database, API, ML, security, infra, versioning, sprints
Stack	Python 3.11 / FastAPI / XGBoost / React 18 / TypeScript / Supabase / Railway / Vercel
Revision policy	All architectural changes require a dated ADR entry. Document version bumped on each release.

Table of Contents

1. **System Vision & Product Architecture** — *Overview, services, data flow, design principles*
2. **System Design — Component Deep Dive** — *SDK, Ingestion API, Feature Worker, ML Service, Dashboard API*
3. **Intelligence Layer** — *Psychology framework, feature engineering, ML ensemble, SHAP, suggestion engine*
4. **Database Design** — *Full schema DDL, indexes, migrations, retention, scaling strategy*
5. **API Contract** — *Every endpoint — request/response, auth, error codes, rate limits*
6. **Frontend Architecture** — *React structure, state, data fetching, semantic search UI*
7. **Machine Learning Pipeline** — *Training, evaluation, retraining, model versioning*
8. **Security & Privacy** — *Auth, RLS, encryption, GDPR, incident response*
9. **Infrastructure & DevOps** — *Environments, CI/CD, Docker, Railway, Vercel, secrets*
10. **Scaling Strategy** — *Per-version scaling plan V1→V3, traffic modeling, ClickHouse migration*
11. **Version Roadmap** — *V1 MVP → V2 Intelligence → V3 Platform with timelines and feature scope*
12. **Epics & Sprint Plan** — *All epics, user stories, story points, acceptance criteria*
13. **Testing Strategy** — *Unit, integration, load, E2E, cross-browser, ML evaluation*
14. **Observability & Monitoring** — *Logging, alerting, Sentry, dashboards, on-call runbook*
15. **Architecture Decision Records** — *Every major technical decision documented*
16. **Glossary** — *Shared vocabulary for the engineering team*

1. System Vision & Product Architecture

The complete picture before any implementation detail

1.1 What Conversiono does

Conversiono is an AI-powered behavioral intelligence platform that sits on top of any e-commerce website. A lightweight JavaScript SDK captures continuous behavioral signals during user sessions — mouse hesitation, scroll patterns, click dynamics, exit intent — and a four-layer intelligence pipeline transforms those signals into named psychological states, real-time abandonment risk scores, and evidence-based intervention recommendations. The result is a platform that tells shop owners not just where users abandoned, but why, and what to do about it.

1.2 The six services

Service	Runtime	Hosting	Primary responsibility
JS SDK	Vanilla JS ES2018+	Merchant CDN	Captures behavioral events, batches, transmits to API
Ingestion API	Python 3.11 / FastAPI	Railway	Validates and stores event batches, triggers async processing
Feature Worker	Python 3.11 / asyncio	Railway (same process at MVP)	Computes feature vectors from raw events
ML Service	Python 3.11 / FastAPI	Railway	Loads trained models, scores sessions, generates SHAP explanations
Dashboard API	Python 3.11 / FastAPI	Railway	Serves merchant analytics: sessions, scores, friction maps, suggestions
Frontend	React 18 / TypeScript	Vercel	Merchant dashboard — friction heatmaps, session explorer, experiment memory

1.3 End-to-end data flow

A user lands on a merchant's shop. The SDK initialises, generates an anonymous UUID stored in sessionStorage, and begins capturing mouse, scroll, and click events at 100ms intervals via requestAnimationFrame. Every two seconds — or when 50 events accumulate — the buffer is flushed as a JSON batch via navigator.sendBeacon to POST /api/v1/events on the Ingestion API.

The Ingestion API validates the SDK key, writes all events in a single bulk INSERT to PostgreSQL, and enqueues a session_update task for the Feature Worker via an asyncio.Queue. The Feature Worker fetches the full session event history, computes all eight feature values, writes the updated session_features record, and calls the ML Service scoring endpoint.

The ML Service — which has all three XGBoost models and the SHAP explainer loaded warm in memory since startup — runs intent classification, friction detection with SHAP attribution, and conversion probability prediction. It cross-references the SHAP output against the psychology framework configuration and returns a SessionIntelligence object. The Feature Worker persists the scores back to the sessions table. The merchant's dashboard polls the Dashboard API every 30 seconds and reflects live updates.

1.4 Core design principles

Principle	What it means in practice
Fail silently on the merchant's site	The SDK must never throw an uncaught error. Every external call is wrapped in try/catch. If the API is unreachable, events are dropped silently.
Bulk operations, never loops	All database writes use bulk INSERT. All reads use indexed queries with explicit LIMIT. No N+1 queries anywhere in the codebase.
Explainability is non-negotiable	Every ML score must be traceable to specific features via SHAP. Every suggestion must cite a psychological mechanism and evidence source.
Privacy by design	No PII collected. Session IDs are anonymous UUIDs. IPs are discarded after country-code extraction. 90-day retention enforced automatically.
Staging first, always	No code reaches production without passing all tests on staging. This applies to hotfixes without exception.
Document decisions, not just code	Every significant architectural choice has an ADR entry. Future engineers must understand why, not just what.

2. Component Deep Dive

Detailed design of every service

2.1 JavaScript SDK

Design constraints

The SDK runs on websites we do not control. Its three non-negotiable constraints are: bundle size under 15KB minified and gzipped, fully asynchronous loading that does not block page render, and silent failure mode that never degrades the host page under any circumstances.

Merchant installation

```
(function(w,d,s){
  w._cvn=w._cvn||function(){(w._cvn.q=w._cvn.q||[]).push(arguments)};
  var js=d.createElement(s); js.async=1;
  js.src='https://cdn.conversiono.io/sdk/v1/cvn.min.js';
  d.head.appendChild(js);
})(window,document,'script');
_cvn('init', { key: 'MERCHANT_SDK_KEY' });
```

Event capture and batching

```
// Internal capture loop — 100ms via requestAnimationFrame
function captureFrame(ts) {
  try {
    buffer.push({
      type:       currentEvent.type,
      timestamp:  ts,
      x:          currentEvent.clientX / window.innerWidth,
      y:          currentEvent.clientY / window.innerHeight,
      velocity:   computeVelocity(currentEvent, prevEvent),
      element_id: getClosestId(currentEvent.target),
    });
    if (buffer.length >= 50) flush();
  } catch(e) { /* silent fail */ }
  requestAnimationFrame(captureFrame);
}

setInterval(flush, 2000);  // also flush every 2 seconds

function flush() {
  if (!buffer.length) return;
  var batch = buffer.splice(0);
  try {
    navigator.sendBeacon(API+'/events',
      JSON.stringify({ session_id:sid, merchant_id:mid, events:batch }));
  } catch(e) {
```

```
    // XHR fallback for older browsers
    fallbackXHR(batch);
  }
}
```

Signals captured

Signal	DOM event	Captured data	Psychological relevance
Mouse movement	mousemove (100ms rAF)	x,y normalised 0-1, velocity px/s, element_id	Primary source for hesitation and anxiety signals
Click	mousedown + mouseup	x,y, element_id, press duration ms	Distinguishes deliberate vs rage clicks
Scroll	scroll (50ms throttle)	scrollY px, direction up/down, velocity px/s	Scroll retreat = decision uncertainty
Exit intent	mouseleave on document	y position, upward velocity	Strongest single abandonment predictor
Visibility change	visibilitychange	hidden/visible, timestamp	Tab switch = comparison shopping or distraction
Session end	beforeunload + pagehide	final timestamp, last page URL	Triggers full ML scoring pipeline

2.2 Ingestion API

Critical implementation rule: bulk insert

```
# routers/events.py
@router.post('/events')
async def ingest_events(batch: EventBatch, db=Depends(get_db)):
    merchant = await validate_sdk_key(batch.merchant_id, db)
    if not merchant:
        raise HTTPException(401, 'Invalid SDK key')

    # ALWAYS bulk insert — never loop individual INSERTs
    await db.execute(
        insert(Event).values([
            {**e.dict(), 'session_id': batch.session_id}
            for e in batch.events
        ])
    )
    await db.commit()
    await enqueue_session_update(batch.session_id)  # non-blocking
    return {'accepted': len(batch.events)}
```

> ⚠ *Never loop individual INSERT statements for events. At 50 events/batch × 500 concurrent sessions = 25,000 rows/second. Single-row inserts will saturate the database immediately.*

2.3 Feature Worker

Feature extraction — all eight features

```
PRICE_KEYWORDS  = ['price','cost','shipping','total','amount','fee']
HESITATION_VEL  = 15.0   # px/s — below this = hesitation
EXIT_Y_THRESH   = 0.05   # top 5% of viewport
EXIT_VEL_MIN    = 50.0   # px/s upward
RAGE_CLICK_WIN  = 2.0    # seconds — clicks within this window
RAGE_CLICK_MIN  = 3      # minimum clicks to qualify as rage

def extract_features(events: list[Event]) -> dict:
    mv = [e for e in events if e.type=='mouse_move']
    cl = [e for e in events if e.type=='click']
    sc = [e for e in events if e.type=='scroll']
    return {
        'hesitation_score':      _hesitation(mv),
        'price_dwell_time_s':    _price_dwell(mv),
        'rage_click_score':      _rage_clicks(cl),
        'scroll_retreat_count':  _scroll_retreats(sc),
        'exit_intent_count':     _exit_intents(mv),
        'checkout_hesitation_s': _checkout_hesitation(events),
        'velocity_variance':     _velocity_variance(mv),
        'session_duration_s':    _duration(events),
    }
```

2.4 ML Service

Model loading and warm cache

```
class MLService:
    def __init__(self):
        base = Path(settings.ML_MODEL_DIR)
        # Load all models at startup — kept in memory permanently
        self.intent     = pickle.load(open(base/'intent_v1.pkl','rb'))
        self.friction   = pickle.load(open(base/'friction_v1.pkl','rb'))
        self.predictor  = pickle.load(open(base/'predictor_v1.pkl','rb'))
        self.explainer  = shap.TreeExplainer(self.friction)
        self.framework  = json.load(open('psychology/framework_v1.json'))
        log('INFO','ml.loaded', models=['intent','friction','predictor'])

    def score(self, features: dict) -> SessionIntelligence:
        vec          = self._to_vector(features)
        intent       = self.intent.predict([vec])[0]
        risk         = float(self.predictor.predict_proba([vec])[0][1])
        shap_vals    = self.explainer.shap_values(np.array([vec]))[0]
        friction_pts = self._interpret_shap(shap_vals, features)
        psych_states = self._map_to_psychology(friction_pts)
        suggestion   = suggestion_engine.recommend(psych_states[0] if
psych_states else None)
        return SessionIntelligence(
            intent=intent, abandonment_risk=round(risk,3),
            friction_points=friction_pts, psych_states=psych_states,
            top_suggestion=suggestion
```

```
)
```

3. The Intelligence Layer

Psychology framework, ML ensemble, SHAP, suggestion engine

3.1 Layer 1 — Psychology Framework

The psychology framework is a versioned JSON configuration that maps observable behavioral patterns to named psychological states grounded in peer-reviewed research. It is the intellectual core of the product and the layer that makes ML output meaningful to non-technical merchants. It is not code — it is knowledge encoded as structured configuration.

Psychological state	Behavioral signature	Research basis	Intervention direction
Loss aversion	Slow mouse near cost fields, scroll retreat, long checkout pause	Kahneman & Tversky Prospect Theory	Reframe cost as saving; anchor against higher reference price
Decision fatigue	Long session, low page progression, multiple revisits to same element	Baumeister ego depletion research	Highlight single recommended option; simplify choice architecture
Decision uncertainty	Repeated scroll direction reversals near checkout	Schwartz Paradox of Choice	Add social proof; reduce visible options; clarify decision path
UI frustration	3+ rapid clicks on same element within 2 seconds	Nielsen Norman broken expectation	Fix interaction design; clarify affordance; add loading state
Abandonment intent	Cursor exits viewport top edge at > 50px/s upward	Mouse tracking exit intent research	Trigger retention intervention; urgency or reassurance messaging
Trust gap	Fast scroll past trust signals (reviews, badges, guarantees)	Fogg persuasion technology	Surface trust signals earlier and more prominently in the flow
Price shock	Sudden hesitation at cost reveal after journey commitment	Ariely predictably irrational anchoring	Reveal total cost earlier; use progressive cost disclosure
Approach-avoidance	Add to cart → remove → re-add cycle	Lewin field theory conflict	Reduce perceived risk; returns policy; guarantee; peer reviews

3.2 Layer 2 — Feature Engineering

Eight features are computed from raw browser events. Each maps directly to one or more psychological states defined in the framework. The mapping between features and psychological states is what makes SHAP output interpretable as human-readable friction diagnoses rather than raw numbers.

Feature	Formula summary	Maps to psychology	Normal range
hesitation_score	avg(pause_duration) × pause_count near cost elements, velocity < 15px/s	Loss aversion, price anxiety	0.0 – 0.3 normal; > 0.6 high risk
price_dwell_time_s	Sum of intervals where cursor within 40px of price/cost element	Price evaluation intensity	< 3s normal; > 8s high friction
rage_click_score	clicks_count / time_on_element where ≥ 3 clicks in 2 seconds	UI frustration	0 = none; > 0.5 severe
scroll_retreat_count	Count of scroll direction reversals during session	Decision uncertainty	< 2 normal; > 5 high friction
exit_intent_count	Count of cursor exits via top of viewport at > 50px/s	Abandonment intent	0 = healthy; ≥ 1 = intervention trigger
checkout_hesitation_s	Duration on checkout pages with no scroll/click activity	Price shock, commitment anxiety	< 10s normal; > 30s critical
velocity_variance	Standard deviation of mouse velocity across full session	Cognitive overload, anxiety level	Low = calm; high = stressed
session_duration_s	last_event.timestamp − first_event.timestamp in seconds	Engagement depth vs fatigue	Category-dependent; use merchant baseline

3.3 Layer 3 — ML Ensemble

Three models, three tasks

The system runs three independent XGBoost models in parallel. They share the same feature vector input but are trained for different tasks, enabling independent retraining, deployment, and evaluation of each without affecting the others.

Model	Task	Output type	Algorithm	Key hyperparameters
Intent classifier	What mode is user in now?	4-class label: browsing / comparing / deciding / exiting	XGBoost multi-class	n_estimators=200, max_depth=5, learning_rate=0.05
Friction detector	Where is friction and what type?	Binary + SHAP feature contributions	XGBoost binary classifier	n_estimators=200, max_depth=6, scale_pos_weight=auto
Conversion predictor	Probability of abandonment in next 2 min	Float 0.0–1.0	XGBoost binary classifier	n_estimators=300, max_depth=6, early_stopping_rounds=20

SHAP explainability — how friction diagnoses are generated

```
# After friction model predicts, SHAP explains WHY
shap_vals = explainer.shap_values(np.array([feature_vector]))[0]

# Map SHAP contributions back to feature names
contributions = sorted(
    zip(FEATURE_NAMES, shap_vals),
    key=lambda x: abs(x[1]),
    reverse=True
)

# Top contributors become friction_points
friction_points = [
    {'feature': f, 'contribution': round(float(v), 3)}
    for f, v in contributions[:3]
    if abs(v) > 0.05  # only significant contributors
]

# Cross-reference with psychology framework
# e.g. hesitation_score → 'loss_aversion' → 'Price anxiety'
psych_states = [framework['feature_to_psychology'][fp['feature']]
                for fp in friction_points]

# Example output:
# friction_points: [{feature:'checkout_hesitation_s', contribution:0.31},
#                   {feature:'exit_intent_count',     contribution:0.24}]
# psych_states:    ['price_shock', 'abandonment_intent']
# dashboard shows: 'Price shock detected at checkout — loss aversion triggered'
```

3.4 Layer 4 — Suggestion Engine

The suggestion engine is deliberately deterministic, not ML-based. It cross-references the friction diagnosis from Layer 3 with an intervention library and the merchant's experiment history to produce ranked, justified recommendations. Every recommendation is traceable to a rule, a psychological mechanism, and a research source — because merchant trust requires explainability.

```
# psychology/intervention_library.json structure
{
  'loss_aversion': {
    'interventions': [
      {
        'id': 'LA-01',
        'title': 'Reframe shipping as savings threshold',
        'description': 'Show "Add €X more for free shipping" instead of flat
fee',
        'predicted_lift_pct': [6, 11],
        'evidence_count': 23,
        'research_ref': 'Ariely 2008 — predictably irrational anchoring'
      },
      {
```

```
        'id': 'LA-02',
        'title': 'Show original price struck through',
        'description': 'Reference price anchoring reduces perceived loss',
        'predicted_lift_pct': [4, 9],
        'evidence_count': 41,
        'research_ref': 'Kahneman 1979 — Prospect Theory loss aversion'
      }
    ]
  }
}
```

4. Database Design

Schema, indexes, migrations, and scaling

4.1 Full DDL — all tables

```
-- ── merchants ──────────────────────────────────────────
CREATE TABLE merchants (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(255) UNIQUE NOT NULL,
  shop_domain   VARCHAR(255) UNIQUE NOT NULL,
  sdk_key_hash  VARCHAR(64)  UNIQUE NOT NULL,  -- SHA-256 only
  plan          VARCHAR(32)  NOT NULL DEFAULT 'trial'
                CHECK (plan IN
('trial','starter','growth','scale','enterprise')),
  is_active     BOOLEAN      NOT NULL DEFAULT true,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- ── sessions ──────────────────────────────────────────
CREATE TABLE sessions (
  id                UUID PRIMARY KEY,  -- set by SDK, anonymous
  merchant_id       UUID NOT NULL REFERENCES merchants(id) ON DELETE CASCADE,
  page_url          TEXT NOT NULL,
  started_at        TIMESTAMPTZ NOT NULL,
  ended_at          TIMESTAMPTZ,
  outcome           VARCHAR(16)  NOT NULL DEFAULT 'unknown'
                    CHECK (outcome IN ('purchase','abandon','unknown')),
  abandonment_risk  FLOAT CHECK (abandonment_risk BETWEEN 0 AND 1),
  friction_score    FLOAT CHECK (friction_score BETWEEN 0 AND 1),
  intent_label      VARCHAR(32),
  country_code      CHAR(2),   -- from IP, IP discarded immediately
  device_type       VARCHAR(16),
  expires_at        TIMESTAMPTZ NOT NULL  -- started_at + 90 days
);

-- ── events ──────────────────────────────────────────
-- HIGH VOLUME: partition by month in production (V2)
CREATE TABLE events (
  id          BIGSERIAL    PRIMARY KEY,
  session_id  UUID         NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  type        VARCHAR(32)  NOT NULL
              CHECK (type IN
('mouse_move','click','scroll','exit_intent','visibility')),
  ts          TIMESTAMPTZ  NOT NULL,
  x           FLOAT,   -- normalised 0-1
  y           FLOAT,   -- normalised 0-1
  velocity    FLOAT,
  element_id  VARCHAR(128),
  metadata    JSONB
);

-- ── session_features ──────────────────────────────────
```

```
CREATE TABLE session_features (
  session_id            UUID PRIMARY KEY REFERENCES sessions(id) ON DELETE
CASCADE,
  hesitation_score      FLOAT,
  price_dwell_time_s    FLOAT,
  rage_click_score      FLOAT,
  scroll_retreat_count  INTEGER,
  exit_intent_count     INTEGER,
  checkout_hesitation_s FLOAT,
  velocity_variance     FLOAT,
  session_duration_s    FLOAT,
  computed_at           TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ── experiments ──────────────────────────────────────────
CREATE TABLE experiments (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  merchant_id      UUID NOT NULL REFERENCES merchants(id) ON DELETE CASCADE,
  title            TEXT NOT NULL,
  hypothesis       TEXT,
  page_element     VARCHAR(128),
  friction_type    VARCHAR(64),
  variant_a        TEXT,
  variant_b        TEXT,
  result           VARCHAR(16) CHECK (result IN
('a_wins','b_wins','no_diff','inconclusive')),
  conversion_delta FLOAT,  -- signed: positive = improvement
  sample_size      INTEGER,
  ran_at           DATE,
  embedding        VECTOR(384),  -- pgvector for semantic search
  source           VARCHAR(32) DEFAULT 'manual',  -- manual/vwo/optimizely/sheets
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ── intervention_results ─────────────────────────────────
CREATE TABLE intervention_results (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  merchant_id     UUID NOT NULL REFERENCES merchants(id) ON DELETE CASCADE,
  experiment_id   UUID REFERENCES experiments(id),
  intervention_id VARCHAR(16) NOT NULL,  -- e.g. 'LA-01'
  triggered_at    TIMESTAMPTZ NOT NULL,
  session_id      UUID REFERENCES sessions(id),
  outcome         VARCHAR(16),
  conversion_delta FLOAT
);
```

4.2 Indexes

```
-- Sessions: merchant dashboard queries
CREATE INDEX idx_sessions_merchant_date  ON sessions(merchant_id, started_at
DESC);
CREATE INDEX idx_sessions_expires        ON sessions(expires_at);
CREATE INDEX idx_sessions_risk           ON sessions(merchant_id,
abandonment_risk DESC);
```

```
-- Events: session detail and feature extraction
CREATE INDEX idx_events_session_ts       ON events(session_id, ts);

-- Experiments: merchant history and semantic search
CREATE INDEX idx_experiments_merchant    ON experiments(merchant_id, ran_at
DESC);
CREATE INDEX idx_experiments_friction    ON experiments(merchant_id,
friction_type);
CREATE INDEX idx_experiments_embed       ON experiments USING ivfflat (embedding
vector_cosine_ops)
  WITH (lists = 100);  -- tune to sqrt(row_count)
```

4.3 GDPR automated retention cleanup

```
-- Runs nightly at 02:00 UTC via Railway cron or pg_cron
-- CASCADE handles events and session_features automatically
DELETE FROM sessions WHERE expires_at < now();

-- expires_at is always set at INSERT time:
INSERT INTO sessions (..., expires_at)
VALUES (..., now() + INTERVAL '90 days');
```

4.4 Database scaling strategy

Scale trigger	Current solution	Migration path	Estimated timeline
Events table > 10M rows	PostgreSQL partitioned by month	Migrate events to ClickHouse; keep sessions/merchants in Postgres	V2 (Month 6–9)
Supabase free tier storage > 400MB	Upgrade to Supabase Pro €25/mo	Already on Pro — no migration needed	Immediate when needed
Semantic search latency > 300ms	pgvector with ivfflat index	Migrate to Pinecone dedicated vector DB	V2 if triggered
Connection pool exhaustion	Supabase pooler (PgBouncer built in)	Increase pool size in Supabase dashboard	No migration needed
> 500 concurrent merchants	Single Postgres instance	Read replicas for analytics queries	V3

5. API Contract

Every endpoint with full request/response specification

Base URL (production)	https://api.conversiono.io/v1
Base URL (staging)	https://staging-api.conversiono.io/v1
Auth — SDK endpoints	X-SDK-Key: {merchant_sdk_key} header
Auth — Dashboard endpoints	Authorization: Bearer {supabase_jwt}
Content type	application/json for all requests and responses
Rate limit — SDK	2,000 requests/minute per SDK key
Rate limit — Dashboard	200 requests/minute per JWT
Versioning	URL-based: /v1, /v2 — old versions maintained for 6 months after deprecation

5.1 SDK Endpoints

POST /events

```
// Request body
{
  'session_id':  'uuid-v4-set-by-sdk',
  'merchant_id': 'uuid-from-sdk-init',
  'events': [
    {
      'type':       'mouse_move',
      'timestamp':  1741203842000,  // Unix ms
      'x':          0.412,          // normalised 0-1
      'y':          0.663,
      'velocity':   24.7,           // px/s
      'element_id': 'checkout-shipping-cost',
      'metadata':   {}
    }
  ]   // max 50 events per batch
}

// Response 200
{ 'accepted': 47, 'session_id': 'uuid', 'status': 'ok' }

// Response 401 — invalid SDK key
{ 'error': 'Invalid SDK key', 'code': 'AUTH_FAILED' }

// Response 422 — validation error
{ 'error': 'events array exceeds limit of 50', 'code': 'VALIDATION_ERROR' }
```

POST /sessions/end

```
// Request body
{
  'session_id': 'uuid',
  'outcome':    'abandon'  // 'purchase' | 'abandon' | 'unknown'
}

// Response 200 — full ML score
{
  'session_id':       'uuid',
  'abandonment_risk': 0.84,
  'intent':           'exiting',
  'friction_points':  [
    { 'feature': 'checkout_hesitation_s', 'contribution': 0.31 },
    { 'feature': 'exit_intent_count',     'contribution': 0.24 }
  ],
  'psych_states':     ['price_shock', 'abandonment_intent'],
  'status':           'scored'
}
```

5.2 Dashboard API Endpoints

Endpoint	Method	Auth	Description
GET /merchants/{id}/sessions	GET	JWT	Paginated session list with filters: date range, min_risk, outcome, page, limit
GET /merchants/{id}/sessions/{sid}	GET	JWT	Full session detail: events, feature vector, SHAP friction, top suggestion
GET /merchants/{id}/analytics/friction-map	GET	JWT	Aggregated friction per DOM element for heatmap overlay
GET /merchants/{id}/analytics/funnel	GET	JWT	Conversion funnel with friction scores per step and drop-off rates
GET /merchants/{id}/analytics/summary	GET	JWT	KPIs: avg risk, total sessions, top friction type, week-on-week change
POST /merchants/{id}/experiments	POST	JWT	Save experiment result — auto-embeds for semantic search
GET /merchants/{id}/experiments	GET	JWT	List experiments with filters: friction_type, result, date range
GET /merchants/{id}/experiments/search	GET	JWT	Semantic search: ?q=natural+language+query — returns ranked results
GET /merchants/{id}/suggestions	GET	JWT	Current top recommendations based on recent friction profile
POST /auth/login	POST	None	Email+password → returns Supabase JWT access and refresh tokens
GET /health	GET	None	Service health: db connected, models loaded, version

5.3 Error codes

HTTP Code	Error code	Meaning	Client action
400	VALIDATION_ERROR	Malformed request body or invalid field values	Fix request per error message
401	AUTH_FAILED	Missing or invalid SDK key or JWT	Re-authenticate or check SDK key
403	FORBIDDEN	Valid auth but insufficient permissions	Check merchant owns the resource
404	NOT_FOUND	Session, merchant, or experiment ID does not exist	Verify IDs
422	PAYLOAD_TOO_LARGE	Events array exceeds 50-item limit	Split batch
429	RATE_LIMITED	Too many requests	Back off exponentially, retry after Retry-After header
500	INTERNAL_ERROR	Unexpected server error — Sentry notified automatically	Retry with backoff; contact support if persists
503	SERVICE_UNAVAILABLE	ML model loading or DB unreachable	Retry after 10 seconds

6. Frontend Architecture

React 18, TypeScript, SWR, semantic search UI

6.1 Project structure

```
dashboard/
├── src/
│   ├── app/
│   │   ├── layout.tsx           // Root layout, auth guard
│   │   ├── page.tsx             // Overview / summary dashboard
│   │   ├── sessions/
│   │   │   ├── page.tsx         // Session explorer with filters
│   │   │   └── [id]/page.tsx    // Session detail: events, SHAP, suggestion
│   │   ├── friction-map/page.tsx// Element-level friction overlay
│   │   ├── experiments/
│   │   │   ├── page.tsx         // Experiment list + semantic search
│   │   │   └── new/page.tsx     // Log new experiment form
│   │   └── settings/page.tsx    // SDK key, integrations, billing
│   ├── components/
│   │   ├── ui/                  // shadcn/ui base components
│   │   ├── FrictionBadge.tsx    // Colour-coded risk score pill
│   │   ├── PsychTag.tsx         // Psychology state label + tooltip
│   │   ├── RiskGauge.tsx        // Radial abandonment risk gauge
│   │   ├── SessionTimeline.tsx  // Scrollable event replay
│   │   ├── SuggestionCard.tsx   // Recommendation card with evidence
│   │   ├── FrictionHeatmap.tsx  // DOM overlay visualisation
│   │   └── SemanticSearch.tsx   // Debounced search with skeleton
│   ├── hooks/
│   │   ├── useSessions.ts       // SWR with 30s polling
│   │   ├── useSessionDetail.ts
│   │   ├── useFrictionMap.ts
│   │   └── useExperiments.ts
│   ├── lib/
│   │   ├── api.ts               // Typed API client (axios + zod)
│   │   ├── auth.ts              // Supabase auth helpers
│   │   └── utils.ts
│   └── types/index.ts           // All shared TypeScript interfaces
├── tailwind.config.ts
├── next.config.ts
└── vercel.json
```

6.2 Data fetching — SWR with polling

```
// hooks/useSessions.ts
import useSWR from 'swr'
import { api } from '@/lib/api'

export function useSessions(merchantId: string, filters: SessionFilters) {
  return useSWR(
```

```
    ['/sessions', merchantId, filters],
    () => api.getSessions(merchantId, filters),
    {
      refreshInterval:   30_000,   // live updates every 30s
      revalidateOnFocus: true,
      dedupingInterval:  5_000,
      onError: (err) => logger.error('sessions.fetch.failed', { err }),
    }
  )
}
```

6.3 Semantic search — debounced with skeleton loading

```
// components/SemanticSearch.tsx
export function SemanticSearch({ merchantId }: Props) {
  const [query, setQuery] = useState('')
  const debouncedQuery    = useDebounce(query, 400)  // 400ms debounce

  const { data, isLoading } = useSWR(
    debouncedQuery.length > 2
      ? ['/experiments/search', merchantId, debouncedQuery]
      : null,   // null key = skip fetch
    () => api.searchExperiments(merchantId, debouncedQuery)
  )

  return (
    <div>
      <Input placeholder='Search past experiments...' onChange={e =>
setQuery(e.target.value)} />
      {isLoading && <SkeletonList count={3} />}
      {data?.map(exp => <ExperimentCard key={exp.id} experiment={exp} />)}
    </div>
  )
}
```

7. Machine Learning Pipeline

Training, evaluation, versioning, retraining

7.1 Training pipeline

```
ml/
├── train.py              # Entry point — trains all 3 models
├── evaluate.py           # AUC, confusion matrix, SHAP summary plots
├── data/
│   ├── load_data.py      # Pulls labeled sessions from Postgres
│   └── preprocess.py     # Feature matrix, train/test split, SMOTE
├── models/
│   ├── intent.py
│   ├── friction.py
│   └── predictor.py
└── config.py             # Hyperparameters, feature names, thresholds
```

Class imbalance handling

```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Apply SMOTE only to training set — NEVER to test set
smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)

# Also set scale_pos_weight in XGBoost:
# scale_pos_weight = count(negative) / count(positive)
neg, pos = np.bincount(y_train)
model = XGBClassifier(scale_pos_weight=neg/pos, ...)
```

AUC gate — block deployment of underperforming models

```
from sklearn.metrics import roc_auc_score

y_pred = model.predict_proba(X_test)[:, 1]
auc    = roc_auc_score(y_test, y_pred)

print(f'AUC: {auc:.3f}')

# Hard gate — CI/CD pipeline fails if AUC below threshold
assert auc >= 0.72, (
```

```
    f'Model AUC {auc:.3f} below production threshold 0.72. '
    f'Do not deploy. Review feature quality and training data.'
)

pickle.dump(model, open(f'models/predictor_v{VERSION}_{DATE}.pkl','wb'))
```

7.2 Model versioning protocol

1. Train and evaluate. AUC must pass 0.72 gate.
2. Name artefact: {model_name}_v{N}_{YYYYMMDD}.pkl
3. Commit artefact to models/ directory with message: 'ml: train predictor_v2 AUC=0.783'
4. Deploy to staging branch. Confirm /health/ml returns new version string.
5. Run integration tests against staging. Verify scored sessions have non-null risk values.
6. Manual approval in GitHub Actions to deploy to production.
7. Monitor Sentry for ML errors for 48 hours post-deploy.
8. If AUC degradation detected: rollback via Railway deployment history.

7.3 Retraining triggers

Trigger	Condition	Action	Urgency
Scheduled	Monthly, regardless of performance	Full retrain on accumulated new sessions	Routine
Performance degradation	Rolling 7-day AUC drops below 0.70	Emergency retrain + investigation	P1 — 48 hours
Data volume	10,000 new sessions since last training	Retrain to capture new patterns	Routine
Feature distribution drift	KL divergence > 0.3 on any feature	Investigate cause + retrain	P1 — investigate first
Merchant category expansion	New vertical added (luxury, B2B, etc.)	Segment-specific model evaluation	Planned

8. Security & Privacy

Authentication, authorisation, GDPR, encryption

8.1 Authentication architecture

```
# Supabase Auth flow for merchant dashboard

# 1. Login → Supabase returns JWT
POST https://xyz.supabase.co/auth/v1/token?grant_type=password
Body: { email, password }
Response: { access_token, refresh_token, expires_in: 3600 }

# 2. All dashboard API calls include Bearer token
Authorization: Bearer eyJ...

# 3. Backend validates via Supabase
async def get_current_merchant(token=Depends(oauth2_scheme)):
    user = supabase.auth.get_user(token)
    if not user:
        raise HTTPException(401, 'Invalid or expired token')
    return user

# SDK key validation (separate from JWT)
def validate_sdk_key(raw_key: str, db) -> Merchant | None:
    key_hash = hashlib.sha256(raw_key.encode()).hexdigest()
    return db.query(Merchant).filter(
        Merchant.sdk_key_hash == key_hash,
        Merchant.is_active == True
    ).first()
```

> ✗ *Never store SDK keys in plaintext. SHA-256 hash on receipt. Show the plaintext key exactly once to the merchant at creation time.*

8.2 Row-level security — complete merchant isolation

```
-- Enable RLS on all tables containing merchant data
ALTER TABLE sessions     ENABLE ROW LEVEL SECURITY;
ALTER TABLE events       ENABLE ROW LEVEL SECURITY;
ALTER TABLE experiments  ENABLE ROW LEVEL SECURITY;

-- Merchants can only see their own rows
CREATE POLICY merchant_own_sessions ON sessions
  FOR ALL USING (merchant_id = auth.uid());

CREATE POLICY merchant_own_experiments ON experiments
  FOR ALL USING (merchant_id = auth.uid());
```

```
-- Backend service role bypasses RLS (used in API services)
-- SUPABASE_SERVICE_KEY must NEVER be sent to the frontend
```

8.3 GDPR compliance checklist

Requirement	Implementation	V1 or V2
No PII collection	Session IDs are anonymous UUIDs. No names, emails, or raw IPs stored.	V1
IP anonymisation	Country code extracted at ingestion, raw IP discarded immediately.	V1
Cookie-free tracking	sessionStorage UUID — no consent banner required under GDPR.	V1
90-day data retention	expires_at set on every session INSERT. Nightly cleanup cron deletes expired rows.	V1
Right to erasure	DELETE /merchants/{id}/data removes all sessions, events, experiments.	V1
Privacy policy published	Available at conversiono.io/privacy before first merchant onboards.	V1
Data Processing Agreement	DPA template provided to every merchant before SDK installation.	V1
Breach notification procedure	72-hour notification to supervisory authority + affected merchants.	V1
Legitimate interest basis	Documented basis for analytics processing in privacy policy.	V1

8.4 Encryption standards

Layer	Method	Notes
In transit	TLS 1.3 — enforced on all endpoints	No HTTP fallback. HSTS header on all responses.
At rest	AES-256 via Supabase managed encryption	Automatic — no configuration needed.
SDK keys	SHA-256 hash	Plaintext never stored, never logged.
JWT signing	RS256 asymmetric	Private key stored in Railway encrypted env vars.
Secrets	Railway + Vercel encrypted environment variables	Never committed to Git. .env.example only.

9. Infrastructure & DevOps

Environments, CI/CD, Docker, secrets, deployment

9.1 Service topology

Service	Provider	Config	Monthly cost (MVP)
Ingestion + Feature Worker	Railway — Python container	512MB RAM, 0.5 vCPU	~€5
ML Service	Railway — Python container	1GB RAM (models in memory), 0.5 vCPU	~€5
Dashboard API	Railway — Python container	512MB RAM, 0.5 vCPU	~€5
PostgreSQL	Supabase — managed Postgres + pgvector	Free tier: 500MB, 2 CPU hours/day	€0
Auth	Supabase Auth	Free tier: 50,000 MAU	€0
Frontend	Vercel — Next.js	Free tier — global CDN	€0
Error monitoring	Sentry	Free tier: 5,000 errors/month	€0
Uptime monitoring	BetterUptime	Free tier: 3 monitors	€0
CI/CD	GitHub Actions	Free tier: 2,000 min/month	€0
Domain + TLS	Namecheap + Let's Encrypt		~€1
TOTAL			~€16/month

9.2 Dockerfile — backend services

```
FROM python:3.11-slim

WORKDIR /app

# Install system deps first (cached layer)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev && rm -rf /var/lib/apt/lists/*

# Install Python deps (cached if requirements.txt unchanged)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
```

```
COPY app/ ./app/
COPY models/ ./models/
COPY psychology/ ./psychology/

# Non-root user for security
RUN useradd -m appuser && chown -R appuser /app
USER appuser

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--
workers", "2"]
```

9.3 CI/CD pipeline — GitHub Actions

```
# .github/workflows/deploy.yml
name: Test → Stage → Prod

on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check .                    # lint
      - run: pytest tests/unit/ -v           # unit tests
      - run: pytest tests/integration/ -v    # integration tests

  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/staging'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: railwayapp/railway-deploy@v2
        with:
          railway-token: ${{ secrets.RAILWAY_STAGING_TOKEN }}

  deploy-production:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production          # requires manual approval
    steps:
      - uses: actions/checkout@v4
      - uses: railwayapp/railway-deploy@v2
        with:
```

```
          railway-token: ${{ secrets.RAILWAY_PROD_TOKEN }}
```

9.4 Environment variables — complete reference

Variable	Services	Description	Example
DATABASE_URL	API, Worker, ML	Async Postgres connection string	postgresql+asyncpg://...@db.supabase.co/postgres
SUPABASE_URL	All backend	Supabase project URL	https://xyzabc.supabase.co
SUPABASE_SERVICE_KEY	All backend	Service role — bypasses RLS — backend ONLY	eyJhbGciOiJIUzI1NiIsInR5cCI...
SUPABASE_ANON_KEY	Frontend only	Public anon key for auth	eyJhbGciOiJIUzI1NiIsInR5cCI...
SENTRY_DSN	All services	Error monitoring endpoint	https://...@sentry.io/12345
ENVIRONMENT	All services	local / staging / production	production
ML_MODEL_DIR	ML Service	Path to .pkl model artefacts	./models
ALLOWED_ORIGINS	API	CORS whitelist	https://app.conversiono.io
LOG_LEVEL	All services	DEBUG in dev, INFO in production	INFO

> ✗ *SUPABASE_SERVICE_KEY is a superuser credential. It bypasses all RLS policies. It must never appear in frontend code, browser DevTools, or public logs.*

10. Scaling Strategy

Traffic modeling, bottleneck analysis, V1 through V3

10.1 Traffic model per version

Version	Merchants	Avg sessions/day	Events/second (peak)	DB write rate (peak)	Primary bottleneck
V1 — MVP	10	5,000	~3 events/s	~150 rows/s	None — well within Railway free tier
V2 — Growth	100	50,000	~30 events/s	~1,500 rows/s	Events table size — add ClickHouse
V3 — Scale	500	250,000	~150 events/s	~7,500 rows/s	ML Service CPU — add horizontal scaling
V3 — Peak (Black Friday)	500	2,000,000	~1,200 events/s	~60,000 rows/s	Full ClickHouse + read replicas + ML autoscaling

10.2 Scaling actions per version

V1
Month 1–3

MVP — Single Process

Zero scaling needed. All services in one Railway project.

- All three API services (Ingestion, ML, Dashboard) run as separate Railway services but on the same starter plan.
- Supabase free tier: 500MB storage, 2 CPU hours/day — sufficient for 10 merchants.
- Feature Worker runs in-process with the Ingestion API via asyncio.Queue — no separate deployment.
- ML models loaded in memory at Railway startup — cold start is the only latency concern.

> *ℹ Railway cold starts take 2–5 seconds. The ML Service keeps itself warm by responding to /health/ml pings from UptimeRobot every 60 seconds, preventing Railway from sleeping the container.*

V2
Month 4–9

Growth — Separation of Concerns

Scale ingestion independently from analytics. Add ClickHouse for events.

- Separate Feature Worker into its own Railway service. Communication via Redis Streams (replace asyncio.Queue). Cost: +€15/mo for Redis on Railway.
- Migrate events table to ClickHouse Cloud free tier. Postgres retains sessions, merchants, features, experiments. ClickHouse handles raw event analytics.
- Upgrade Railway services to Hobby plan (512MB → 1GB RAM) for ML Service to handle increased model complexity.
- Add database connection pooling explicitly: PgBouncer already included in Supabase — increase pool size to 25 in dashboard.
- Add Supabase Pro (€25/mo) for increased storage and daily backups.

V3
Month 10–18

Scale — Production Grade

Horizontal scaling, read replicas, dedicated ML infrastructure.

- ML Service: add horizontal scaling on Railway (2 replicas behind load balancer). Alternatively migrate ML serving to AWS Lambda with model stored in S3.
- Ingestion API: scale to 3 replicas with Railway's auto-scaling on CPU > 70%.
- PostgreSQL: add Supabase read replica for analytics queries. Dashboard API reads from replica; all writes to primary.
- ClickHouse: upgrade from free tier to ClickHouse Cloud Production for SLA and GDPR data residency guarantees.
- Semantic search: migrate from pgvector to Pinecone if experiment corpus exceeds 100,000 entries per merchant.
- CDN for SDK: move from self-hosted Railway to CloudFlare CDN for global low-latency SDK delivery.

10.3 Database scaling decision tree

Condition	Metric	Action
Supabase DB > 400MB	Dashboard alert	Upgrade to Supabase Pro (€25/mo) — immediate
Events table > 10M rows	Query latency > 200ms	Migrate events to ClickHouse — V2 planned
DB connection errors in logs	Pool exhaustion	Increase PgBouncer pool in Supabase dashboard
Semantic search p95 > 300ms	APM metric	Re-tune ivfflat lists parameter; escalate to Pinecone at 100k rows
ML Service p95 > 800ms	Sentry performance	Add second Railway replica or migrate to Lambda
Ingestion API CPU > 80%	Railway metrics	Scale to 2 replicas; investigate bulk insert query plan

11. Version Roadmap

V1 MVP through V3 Platform with feature scope and timelines

V1
Weeks 1–8 (Months 1–2)

MVP — Core Intelligence

Prove the core thesis: will merchants install the SDK and find friction data valuable?

V1 Feature scope

- JavaScript SDK: 5 behavioral signals, < 15KB bundle, async loading, sessionStorage UUID
- Ingestion API: event batching, bulk INSERT, SDK key validation
- Feature Worker: all 8 features computed in real time
- ML Service: three XGBoost models (intent, friction, predictor) with SHAP explainability
- Psychology framework v1: 8 psychological states encoded
- Dashboard: session list, friction scores, basic heatmap, session detail with SHAP labels
- Authentication: Supabase Auth for merchants
- Infrastructure: Railway + Supabase + Vercel + Sentry + GitHub Actions
- GDPR: 90-day retention, cookie-free tracking, privacy policy published

V1 Success criteria

- ML model AUC > 0.72 on held-out test set before any merchant onboarding
- SDK page load impact < 50ms measured on a standard Shopify store
- API p95 latency < 200ms on /events endpoint under 100 concurrent sessions
- 10 pilot merchants onboarded with real session data flowing
- At least 7/10 pilot merchants rate friction data as 'actionable' in qualitative interview

V2
Weeks 9–20 (Months 3–5)

Intelligence — Experiment Memory & Suggestion Engine

Build the compounding moat: experiment memory, semantic search, suggestion engine.

V2 Feature scope

- Psychology framework expanded to 12 psychological states
- Friction detector upgraded with expanded intervention library (50+ interventions)
- Experiment memory: merchants log A/B tests with automatic psychological tagging
- Semantic search: natural language search over experiment history via pgvector
- External integrations: VWO webhook receiver, Optimizely result sync, Google Sheets import
- Suggestion engine v1: deterministic rules-based recommendations with evidence citations

- Funnel analytics: conversion funnel with friction scores per step
- Friction map overlay: visual heatmap of friction intensity per DOM element
- ML models retrained on real merchant data from V1 pilots — replace bootstrap dataset
- Infrastructure: Feature Worker separated to own service, Redis Streams, ClickHouse migration
- First paying merchants: self-serve onboarding, billing via Stripe

V2 Success criteria

- ML model AUC > 0.78 with real merchant training data
- Semantic search p95 < 300ms
- 20 paying merchants at average €149/month
- €3,000 MRR by end of Month 5

V3
Weeks 21–52 (Months 6–12)

Platform — Scale & Series A Readiness

Scale to 80+ merchants, Shopify App Store, enterprise features.

V3 Feature scope

- Shopify App Store listing: self-serve installation for 4.8M+ Shopify merchants
- WooCommerce plugin: WordPress plugin for one-click SDK installation
- Real-time intervention triggers: API webhook fired when abandonment_risk > threshold
- Merchant-specific model fine-tuning: models retrained per merchant segment at scale
- Advanced analytics: cohort analysis, segment comparison, A/B test significance calculator
- Enterprise tier: dedicated model, SLA, custom integrations, SSO
- Mobile SDK: React Native and iOS/Android native SDK for mobile commerce
- Infrastructure: horizontal ML scaling, Postgres read replica, CloudFlare CDN for SDK
- Series A metrics package: automated MRR reporting, churn analytics, investor dashboard

V3 Success criteria

- 80 paying merchants, €25,000 MRR by end of Month 12
- NPS > 40 across paying merchant base
- Shopify App Store listing live with > 50 installs in first 30 days
- Series A deck prepared, investor conversations initiated

12. Epics & Sprint Plan

All epics, user stories, story points, acceptance criteria

Sprints are two weeks. Story points: 1=trivial, 2=simple, 3=moderate, 5=complex, 8=very complex, 13=spike. Labels: BE=backend, FE=frontend, ML=machine learning, INF=infrastructure, SEC=security, QA=testing. Acceptance criteria follow each story in brackets.

Epic 1 — Data Foundation & ML Baseline

Sprint 1 — Weeks 1–2 | V1

- **[INF]** Set up monorepo with Python backend, React frontend, ML directory *2sp*
- **[INF]** Configure Supabase project — run all DDL migrations via Alembic *3sp*
- **[INF]** GitHub Actions: lint + unit tests + staging auto-deploy pipeline *3sp*
- **[INF]** Sentry error monitoring on all backend services *1sp*
- **[INF]** BetterUptime monitors on /health endpoints *1sp*
- **[ML]** Load labeled session data into Postgres (UCI dataset or existing data) *2sp*
- **[ML]** Implement all 8 feature extraction functions with tests *5sp*
- **[ML]** Train conversion predictor XGBoost baseline — AUC > 0.70 *5sp*
- **[ML]** Write psychology framework JSON v1 — 8 psychological states *5sp*

Epic 2 — ML Ensemble + SHAP

Sprint 2 — Weeks 3–4 | V1

- **[ML]** Train intent classifier — 4 classes, evaluate confusion matrix *5sp*
- **[ML]** Train friction detector — binary classifier with SHAP explainer *8sp*
- **[BE]** Implement MLService class: load 3 models warm at startup *5sp*
- **[BE]** Implement _interpret_shap: map contributions to psychology framework *5sp*
- **[BE]** POST /api/v1/sessions/score endpoint with full SessionIntelligence output *3sp*
- **[QA]** Unit tests: MLService returns valid scores and SHAP attributions *3sp*
- **[ML]** Evaluate all 3 models: AUC reports and SHAP summary plots *2sp*
- **[INF]** Document ADR-003: three models decision *1sp*

Epic 3 — JavaScript SDK

Sprint 3 — Weeks 5–6 | V1

- **[BE]** Event capture loop: 5 signal types, 100ms rAF, x/y/velocity/element_id *5sp*
- **[BE]** Session UUID: sessionStorage generation, merchant init, SDK key embedding *3sp*
- **[BE]** Batching: flush at 50 events or 2-second interval whichever comes first *3sp*
- **[BE]** navigator.sendBeacon transport with XHR fallback for older browsers *2sp*

[BE] Silent fail: every external call in try/catch, console.warn in dev only *2sp*

[INF] Minification pipeline — verify final bundle < 15KB gzipped *2sp*

[QA] Cross-browser test: Chrome, Firefox, Safari, Edge, Mobile Chrome, Safari iOS *3sp*

[QA] SDK integration test against staging Ingestion API — verify data flows *3sp*

Epic 4 — Backend API Complete

Sprint 4 — Weeks 7–8 | V1

[BE] POST /events: bulk INSERT, SDK key validation, asyncio queue enqueue *5sp*

[BE] POST /sessions/end: trigger Feature Worker + full ML scoring *5sp*

[BE] Feature Worker: stateful session object, recompute all 8 features per batch *5sp*

[BE] GET /sessions: paginated, filtered by date/risk/outcome, RLS enforced *3sp*

[BE] GET /sessions/{id}: full detail — events, features, SHAP, suggestion *3sp*

[BE] GET /analytics/friction-map: aggregate by element_id with avg hesitation *3sp*

[BE] GET /analytics/funnel: step-by-step with drop-off rates and friction scores *3sp*

[SEC] Rate limiting (slowapi): 2000 req/min SDK, 200 req/min dashboard *2sp*

[SEC] Input validation: strict Pydantic models on all endpoints *2sp*

[QA] Integration tests: full session pipeline ingest → feature → score → retrieve *5sp*

[QA] Load test (Locust): 500 users, verify /events p95 < 200ms *3sp*

Epic 5 — Merchant Dashboard v1

Sprint 5 — Weeks 9–10 | V1

[FE] Scaffold Next.js app: routing, Supabase Auth integration, API client (axios + zod) *3sp*

[FE] Overview page: session count, avg risk, top friction types, 7-day trend *5sp*

[FE] Sessions list: table with filters, FrictionBadge, PsychTag, SWR 30s polling *5sp*

[FE] Session detail: feature values, SHAP friction labels, SuggestionCard *8sp*

[FE] FrictionBadge component: green/amber/red colour coding by risk threshold *2sp*

[FE] PsychTag component: psychology state chip with tooltip citing research *2sp*

[FE] RiskGauge component: radial gauge for abandonment risk *3sp*

[FE] Responsive layout: desktop + tablet — minimum 1024px width *2sp*

[FE] Deploy to Vercel staging — smoke test all pages *1sp*

Epic 6 — Security Hardening

Sprint 6 — Weeks 11–12 | V1

[SEC] GDPR cleanup cron: nightly DELETE sessions WHERE expires_at < now() *2sp*

[SEC] SDK key hashing: SHA-256 on receipt, plaintext shown once only *2sp*

[SEC] RLS policies on sessions, events, experiments tables *3sp*

[SEC] CORS: restrict to known origins in staging and production *1sp*

[SEC] Error sanitisation: no stack traces in API responses *2sp*

[SEC] OWASP ZAP automated scan on staging — fix all high and medium findings *5sp*

[SEC] Privacy policy published on conversiono.io/privacy *2sp*

[SEC] DPA template drafted for merchant agreements *2sp*

[INF] Production environment on Railway — separate from staging *3sp*

[SEC] Secrets audit: verify no secrets in git history *2sp*

Epic 7 — First Merchant Onboarding

Sprint 7 — Weeks 13–14 | V1

[BE] Merchant sign-up flow: email → Supabase Auth → SDK key generation *5sp*

[FE] SDK installation guide: step-by-step for Shopify and WooCommerce *2sp*

[BE] Health check endpoints: /health, /health/db, /health/ml with version *2sp*

[BE] Onboard Pilot Merchant 1 — manual support, watch all data live *5sp*

[ML] Validate ML scores against merchant's known conversion rates *3sp*

[BE] Qualitative interview: is friction data actionable? Collect verbatim feedback *2sp*

[BE] Fix top 5 issues from Pilot Merchant 1 session *5sp*

[BE] Onboard Pilot Merchants 2 and 3 *3sp*

[INF] Update ADRs with any architectural decisions made during pilots *1sp*

Epic 8 — Experiment Memory & Integrations

Sprint 8 — Weeks 15–16 | V2

[BE] Add VECTOR(384) column to experiments, run Alembic migration *2sp*

[BE] Integrate sentence-transformers: encode experiment on save *3sp*

[BE] POST /experiments: save + auto-embed with friction type auto-tagging *5sp*

[BE] VWO webhook receiver: parse and auto-import test results *5sp*

[BE] Google Sheets import: parse experiment CSV from shared sheet *3sp*

[FE] Experiments page: list, create form, filter by friction type *5sp*

[FE] ExperimentCard component: result badge, conversion delta, friction tag *3sp*

[QA] Unit tests: experiment save creates valid embedding *2sp*

Epic 9 — Semantic Search + Suggestion Engine

Sprint 9 — Weeks 17–18 | V2

[BE] GET /experiments/search: pgvector cosine similarity search endpoint *5sp*

[FE] SemanticSearch component: debounced input (400ms), skeleton loading *3sp*

[BE] Suggestion engine v1: intervention library (30 interventions), rules matching *8sp*

[FE] SuggestionCard: title, evidence, predicted lift range, research citation *3sp*

[BE] GET /suggestions endpoint: top 3 recommendations based on current friction profile *3sp*

[ML] Intervention library JSON: 30 entries across 8 psychological states *5sp*

[QA] Unit tests: suggestion engine returns relevant interventions for each friction type *3sp*

[QA] Performance test: semantic search p95 < 300ms with 1,000 experiment records *2sp*

Epic 10 — Infrastructure Scale + Shopify Listing

Sprint 10 — Weeks 19–24 | V3

[INF] Separate Feature Worker to own Railway service with Redis Streams *8sp*

[INF] ClickHouse Cloud setup — migrate events table from Postgres *8sp*

[BE] Shopify App Store submission: app listing, OAuth install flow, review compliance *13sp*

[BE] WooCommerce plugin: PHP wrapper that injects SDK on WordPress sites *8sp*

[BE] Stripe billing integration: subscription management, plan enforcement *8sp*

[BE] Real-time intervention webhook: fire when abandonment_risk > merchant threshold *5sp*

[FE] Advanced analytics: cohort analysis, week-over-week friction trends *8sp*

[INF] Railway horizontal scaling: 2 replicas for ML Service and Ingestion API *3sp*

[INF] Supabase Pro upgrade + read replica for Dashboard API analytics queries *2sp*

Sprint velocity summary

Sprint	Weeks	Theme	Est. story points	Key deliverable
1	1–2	Data + ML baseline	33	Feature pipeline + AUC > 0.70
2	3–4	ML ensemble + SHAP	32	3 models + SHAP + psychology framework
3	5–6	JavaScript SDK	23	< 15KB SDK, cross-browser tested
4	7–8	Backend API	39	Full pipeline: events → features → scores → API
5	9–10	Dashboard v1	31	Merchant sees sessions and friction live
6	11–12	Security hardening	24	GDPR, RLS, OWASP scan passed
7	13–14	First merchant	28	Real sessions, qualitative validation
8	15–16	Experiment memory	28	Merchants log and import A/B tests
9	17–18	Semantic search + suggestions	32	Natural language search + recommendations
10	19–24	Scale + Shopify	63	App Store live, infrastructure production-grade
TOTAL			333	V1 → V2 complete + V3 initiated

13. Testing Strategy

Unit, integration, load, E2E, cross-browser, ML evaluation

13.1 Test pyramid

Level	Coverage target	Tool	Run when	Who owns
Unit tests	≥ 80% of business logic	pytest	Every commit (CI — blocks merge)	Every engineer
Integration tests	All API endpoints + full pipeline	pytest + httpx	Every PR merge	Backend engineer
Load tests	API under 500 concurrent users	Locust	Monthly + before every major release	Tech co-founder
End-to-end tests	Critical user flows in browser	Playwright	Before each production deploy	Frontend engineer
Cross-browser SDK	Chrome, Firefox, Safari, Edge, Mobile	BrowserStack free	Before every SDK version release	Tech co-founder
ML evaluation	AUC, precision, recall, SHAP plots	pytest + sklearn	Every model training run (CI gate)	ML engineer

13.2 Critical unit tests — must exist before V1 launch

```
# Feature engine
test_hesitation_zero_when_no_price_elements()
test_hesitation_positive_near_cost_element()
test_rage_clicks_detected_at_threshold()
test_scroll_retreat_count_accuracy()
test_exit_intent_detected_above_threshold()

# ML Service
test_model_loads_without_error()
test_score_returns_float_between_0_and_1()
test_shap_returns_top_3_contributors()
test_psychology_framework_maps_all_features()

# Suggestion engine
test_loss_aversion_maps_to_LA_interventions()
test_returns_no_duplicate_intervention_ids()
test_excludes_previously_tested_interventions()

# GDPR
test_session_expires_at_set_to_90_days()
test_cleanup_deletes_expired_sessions()
test_cleanup_cascades_to_events()
```

13.3 Load test targets — must pass before first merchant

Endpoint	Concurrent users	Target p50	Target p95	Max error rate
POST /events	500	< 30ms	< 150ms	< 0.1%
POST /sessions/end (full ML)	100	< 100ms	< 400ms	< 0.5%
GET /sessions (dashboard)	50	< 60ms	< 200ms	< 0.1%
GET /friction-map	50	< 100ms	< 300ms	< 0.1%
GET /experiments/search (semantic)	20	< 80ms	< 300ms	< 0.1%

14. Observability & Monitoring

Logging, alerting, runbook, on-call procedures

14.1 Structured logging

```
import logging, json
from datetime import datetime, timezone

def log(level: str, event: str, **ctx):
    print(json.dumps({
        'ts':      datetime.now(timezone.utc).isoformat(),
        'level':   level,
        'event':   event,
        'service': settings.SERVICE_NAME,
        'env':     settings.ENVIRONMENT,
        **ctx
    }))

# Examples
log('INFO',  'events.batch.received', session_id=sid, count=47, latency_ms=18)
log('WARN',  'ml.score.slow',         session_id=sid, latency_ms=620)
log('ERROR', 'db.insert.failed',      session_id=sid, error=str(e))
```

14.2 Alerting rules

Alert name	Condition	Severity	Response SLA	Who is paged
API down	UptimeRobot: /health non-200 for > 60s	P0 — critical	Page immediately, fix within 15 min	Tech co-founder
High error rate	Sentry: > 5% of requests → 5xx in 5 min	P0 — critical	Page immediately, fix within 30 min	Tech co-founder
ML scoring failures	Sentry: > 10 ML errors in 10 min	P1 — high	Fix within 2 hours	ML engineer
API latency spike	p95 /events > 500ms for 10 requests	P1 — high	Investigate within 2 hours	Backend engineer
DB storage > 400MB	Supabase dashboard alert	P2 — medium	Upgrade within 24 hours	Tech co-founder
AUC degradation	Rolling 7-day AUC < 0.70	P2 — medium	Retrain within 48 hours	ML engineer
Disk space (Railway)	Container disk > 80%	P2 — medium	Clean up within 24 hours	Tech co-founder

14.3 Production runbook — incident procedures

Incident type 1: API returning 500 errors

9. Open Sentry — identify the error and stack trace.
10. Check Railway → Service → Logs for the last 500 error context.
11. If database connection error: check Supabase dashboard for connection pool exhaustion.
12. If code error: identify the offending commit in GitHub.
13. Rollback: Railway → Service → Deployments → click Rollback on last green deployment.
14. Verify /health returns 200 within 90 seconds.
15. Notify affected merchants if downtime exceeded 5 minutes.

Incident type 2: ML scoring stopped updating

16. Verify sessions are showing null abandonment_risk in Supabase dashboard.
17. Check Railway → ML Service → Logs for model loading errors.
18. Run: curl https://api.conversiono.io/health/ml — verify model version in response.
19. If model file missing: re-deploy from last successful deployment in Railway.
20. If feature extraction bug: sessions will rescore automatically when fixed.
21. Fall back to rule-based scoring as temporary measure (config flag: USE_RULES_FALLBACK=true).

Incident type 3: SDK causing merchant site slowdown

22. Confirm with merchant: does removing the SDK snippet restore normal performance?
23. Check Railway logs for slow /events responses.
24. If DB bottleneck: check Supabase slow query log.
25. Deploy hotfix: increase batch interval from 2s to 5s (config update, no code change).
26. Reassure merchant with timeline and root cause before end of business day.

15. Architecture Decision Records

Every major technical decision with context and consequences

Every significant architectural decision in this document is captured here. Future engineers must understand why decisions were made, not just what they are. When a new architectural decision is made, add a new ADR with today's date.

ADR-001 — asyncio.Queue instead of Kafka for MVP event pipeline

Date	March 2026
Status	Accepted
Context	Feature Worker must receive session update notifications from the Ingestion API. Options range from in-process asyncio.Queue to Redis Streams to full Kafka.
Decision	Use Python asyncio.Queue for in-process messaging at MVP scale.
Consequences	Zero infrastructure cost, zero operational complexity. Hard limit: only works while Ingestion API and Feature Worker run in the same process. Migration trigger: when services need independent scaling (approximately V2, Month 6).
Migration path	Replace asyncio.Queue with Redis Streams on Railway. Redis adds ~€15/month.

ADR-002 — pgvector on Supabase instead of dedicated vector database

Date	March 2026
Status	Accepted
Context	Experiment memory requires semantic search. Options: Pinecone, Weaviate, Qdrant, or pgvector on existing Postgres.
Decision	pgvector on Supabase with ivfflat index. Keeps the entire data layer in one system.
Consequences	Acceptable at MVP scale (< 10,000 experiments per merchant). Query latency degrades above ~100,000 vectors. Migration trigger: semantic search p95 exceeds 300ms in production.
Migration path	Export embeddings to Pinecone. Application change is minimal — swap the search function.

ADR-003 — Three independent XGBoost models instead of one multi-output model

Date	March 2026
Status	Accepted
Context	The ML layer needs to predict intent class, friction attribution, and abandonment risk. Options: single multi-output model or three independent models.
Decision	Three independent XGBoost models, each trained for exactly one task.
Consequences	Each model can be retrained, deployed, and evaluated independently. Easier to debug. SHAP works cleanly with single-output models. Cost: three artefacts to maintain. Migration trigger: if maintaining three training pipelines becomes a bottleneck after Series A.

ADR-004 — Deterministic rules-based suggestion engine, not ML

Date	March 2026
Status	Accepted
Context	The suggestion engine could be a learned recommendation model or a deterministic rules-based system.
Decision	Deterministic rules-based system with curated intervention library.
Consequences	Every recommendation is fully explainable and traceable. Merchants trust recommendations they can understand. Limitation: cannot learn from outcomes automatically. Migration trigger: when intervention library exceeds 200 entries and enough outcome data has accumulated to train a recommendation model (approximately V3).

ADR-005 — navigator.sendBeacon instead of WebSocket for SDK transport

Date	March 2026
Status	Accepted
Context	The SDK needs a reliable mechanism to send event batches to the API, including on page unload.
Decision	navigator.sendBeacon as primary transport. XHR synchronous fallback for browsers without Beacon support.
Consequences	Beacon is the only transport that fires reliably on page unload, making it essential for session end detection. WebSocket would require a persistent connection and add significant SDK complexity. Beacon browser support is 97%+ globally.

ADR-006 — sessionStorage UUID instead of cookies for session identity

Date	March 2026
Status	Accepted
Context	The SDK needs to identify a session across page navigations. Options: first-party cookie, fingerprint, sessionStorage UUID.
Decision	sessionStorage UUID generated by the SDK on initialisation.
Consequences	No GDPR consent banner required — sessionStorage is not persistent tracking under GDPR ePrivacy guidance. UUID is automatically cleared when the browser tab closes, which correctly ends the session. Limitation: if the user opens a new tab, a new session UUID is generated — this is acceptable for our granularity.

16. Glossary

Shared vocabulary for the engineering team

Term	Definition
AUC-ROC	Area Under the Receiver Operating Characteristic Curve. Primary ML evaluation metric. 0.5 = random classifier. 1.0 = perfect. Production threshold: > 0.72 for V1, > 0.78 for V2.
SHAP	SHapley Additive exPlanations. A framework for explaining individual ML predictions by computing each feature's marginal contribution to the output. Used to produce human-readable friction diagnoses.
Feature vector	A fixed-length array of floating-point numbers representing one session's behavioral state. The input to all three ML models. Length: 8 features at V1.
SessionIntelligence	The output object from the ML Service containing intent label, abandonment_risk score (0–1), friction_points with SHAP contributions, mapped psychological states, and top suggestion.
Psychology framework	The versioned JSON configuration that maps behavioral features to named psychological states (loss aversion, decision fatigue, etc.) grounded in academic research. Version-controlled in psychology/framework_v{N}.json.
Suggestion engine	The deterministic rules-based system that cross-references friction diagnoses with the intervention library and merchant experiment history to produce ranked, justified A/B test recommendations.
Intervention library	A curated JSON database of psychology-backed A/B test suggestions organised by friction type. Each entry includes predicted conversion lift range, evidence count, and research citation.
ADR	Architecture Decision Record. A short document capturing a significant technical decision — context, options considered, decision made, consequences, and migration trigger.
RLS	Row-Level Security. A PostgreSQL/Supabase feature enforcing that authenticated users can only read and write rows belonging to their own merchant account.
Semantic search	Search using natural language meaning rather than exact keyword matching. Implemented via sentence-transformer embeddings stored as VECTOR(384) columns, queried with pgvector cosine similarity.
Exit intent	Behavioral signal detected when the user's cursor exits the document viewport via the top edge at > 50px/s upward velocity. Strongest single-feature predictor of session abandonment.
Rage click	Three or more clicks on the same DOM element within a two-second window. Indicates UI frustration caused by broken interaction design or an unresponsive element.
Hesitation score	Computed feature: average pause duration × pause frequency near price/cost DOM elements, measured when mouse velocity drops below 15px/s. Primary indicator of loss aversion and price anxiety.
Bulk INSERT	The practice of inserting multiple database rows in a single SQL statement. Required for all event writes in Conversiono to avoid database saturation under load.

Cold start	The latency experienced when Railway spins up a sleeping container. Mitigated by keeping services warm via UptimeRobot /health pings every 60 seconds.
p95 latency	The 95th percentile response time — 95% of requests complete faster than this value. The primary latency SLA metric used throughout this document.
SMOTE	Synthetic Minority Over-sampling Technique. Applied to training data to address class imbalance in e-commerce datasets (typically 75–85% abandonment). Never applied to the test set.
DPA	Data Processing Agreement. The legal contract between Conversiono (data processor) and each merchant (data controller) required under GDPR Article 28.